SIMATS ENGINEERING

CO5-ASSESSMENT TOOL3-Reverse Engineering

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192312428**
Name	**B.Bhavana**

COURSE OUTCOME COVERED

CO5: *Design intelligent agents and real-time AI-based systems (chatbots, expert systems, emotion detection, edge AI, autonomous drones, and related applications) by selecting appropriate agent architectures, knowledge representation, and reasoning mechanisms to solve real-world problems. (BL6)*

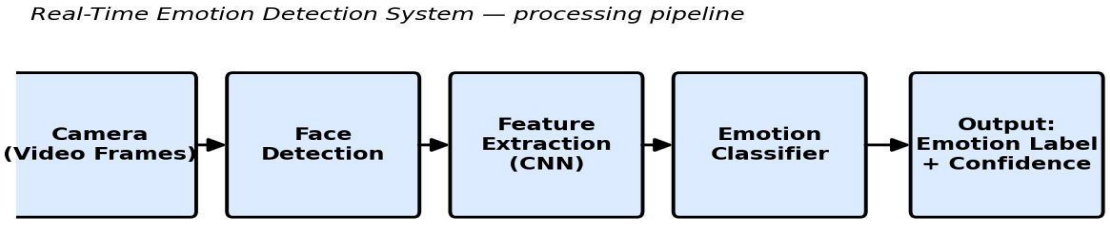
Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:50**

Q1. The following is a conversation between a user and an AI-based flight-booking chatbot. Study the transcript given below and reverse-engineer the chatbot's design by: (a) writing its PEAS description, (b) classifying the type of agent it represents with justification, (c) identifying the likely component (NLU, Dialogue Manager, Knowledge Base, or NLG) responsible for each chatbot response, and (d) suggesting one enhancement using a learning-agent architecture.

Speaker	Message
User	I want to book a flight to Chennai next Friday.
Chatbot	Sure! What time would you like to depart, and do you have a preferred airline?
User	Morning flight, no preference.
Chatbot	I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?

Q2. The block diagram below shows the processing pipeline of a Real-Time Emotion Detection System. Reverse-engineer the system by: (a) explaining the function performed at each stage of the pipeline, (b) identifying the likely AI/ML technique used at each stage, (c) writing the PEAS description of the system, and (d)



classifying the agent type (e.g., simple reflex, model-based) with justification.

Q3. An Expert System Shell for medical diagnosis contains the rule base, input facts, and reasoning trace given below. Study it and reverse-engineer the shell by: (a) identifying the type of chaining (forward/backward) used to reach the diagnosis, with justification, (b) identifying the expert-system-shell components illustrated (knowledge base, inference engine, explanation facility), (c) explaining how the shell would perform knowledge acquisition to add a new rule for dengue, and (d) explaining, with reasoning, whether backward chaining could reach the same diagnosis.

Rule	IF (condition)	THEN (conclusion)
R1	fever = yes AND cough = yes	suspect = flu
R2	suspect = flu AND body_ache = yes	diagnosis = Influenza
R3	fever = yes AND rash = yes	suspect = measles

Given facts: fever = yes, cough = yes, body_ache = yes
Reasoning trace: R1 fires first (fever & cough matched) → suspect = flu. R2 then fires (suspect = flu & body_ache matched) → diagnosis = Influenza.
Explanation generated by the shell: *"Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza."*

Q4. The diagram below shows the system architecture of an Autonomous Drone used for Disaster Detection with Edge AI deployment. Reverse-engineer the design by: (a) writing the PEAS description of the drone agent, (b) classifying the agent type (goal-based/utility-based) with justification, (c) identifying which components run on the edge versus the cloud and justifying why on-edge processing was chosen for detection and obstacle avoidance, and (d) identifying whether the agent's control architecture is reactive, deliberative, or hybrid, with justification.

Q5. The table below shows sample output logs from an AI-based Deepfake & Face Authenticity Verification system across five video frames. Study the output and reverse-engineer the system by: (a) identifying the likely input features used by the model (e.g., blink rate, texture/artifact score), (b) identifying the type of model/technique likely used to produce these outputs, (c) explaining how the final Real/Fake decision may have been derived from the frame-level confidence scores,

and (d) suggesting one way a swarm/ensemble-based approach could improve the system's reliability.

Frame No.	Blink Rate (per 10s)	Texture/Artifact Score	Fake Confidence (%)	Prediction
1	4	0.12	8	Real
2	1	0.61	74	Fake
3	0	0.83	91	Fake
4	5	0.09	5	Real
5	2	0.55	68	Fake

Overall system verdict: Fake (3 of 5 frames flagged — majority vote)
Code of Conduct:

I B.Bhavana(192312428)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Bhavana

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Identification of Agent Type & PEAS (correctly inferred from the given artifact)	Correctly identifies the agent type and gives a complete, accurate PEAS description, fully consistent with the given artifact.	Identifies the agent type and PEAS description correctly with only minor omissions.	Partially correct identification of agent type/PEAS; some elements are missing or inconsistent with the artifact.	Agent type/PEAS identification is largely incorrect or not attempted.	**10**
Identification of Architecture & Components (mapping artifact evidence to system components)	Accurately identifies all relevant architectural components and correctly maps each to specific evidence in the artifact.	Identifies most components correctly with reasonable mapping to the artifact; minor gaps present.	Identifies some components; mapping to the artifact is weak or partially incorrect.	Little or no correct identification of architecture/components.	**10**
Identification of Underlying Technique/Algorithm/Reasoning	Correctly reverse-engineers the technique, algorithm, or	Correctly identifies the technique/ mechanism	Identification is only partially correct or lacks	Technique/mechanism is incorrectly identified or not addressed.	**10**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Mechanism	reasoning mechanism (e.g., chaining type, ML technique) with clear supporting evidence.	with adequate, mostly accurate justificatio n.	sufficient supporting evidence.		
Quality of Justification & Use of Evidence (linking conclusions back to the given artifact)	Every conclusion is clearly justified with specific, accurate references to the given artifact.	Most conclusions are justified with reasonable references to the artifact.	Justification is vague or only loosely connected to the artifact.	Conclusions are stated with little or no justification or evidence.	**10**
Clarity & Completeness of Presentation (addresses all sub-parts of the question in an organised manner)	All sub-parts are addressed completely, in a clear, well-organised, and logically structured manner.	Most sub-parts are addressed clearly with minor gaps in organisatio n or completene ss.	Some sub-parts are missing or the presentation is unclear/poor ly organised.	Response is incomplete, disorganised, or largely missing.	**10**
				Total	50

Question 1: Flight-Booking Chatbot

(a) PEAS Description

- Performance measure: successful flight bookings completed, accuracy of understood intent/slots, minimum number of dialogue turns, fast response time, user satisfaction.
- Environment: online flight-booking platform; natural-language text input from users; live, changing flight inventory/prices; partially observable (the user's full travel intent is not known until asked) and dynamic (availability changes in real time).
- Actuators: text responses to the user, display of flight lists/options, booking confirmation actions.
- Sensors: the text typed by the user, session/conversation history, previously filled slots (destination, date, time, airline).

(b) Agent Type and Justification

The chatbot is best classified as a goal-based agent with utility-based ranking, built on a model-based (slot-filling) core.

- Model-based, because it must remember what has already been said (destination = Chennai, date = next Friday) across turns - a simple reflex agent has no memory of prior utterances.
- Goal-based, because its objective is clearly defined: gather all required slots and complete a valid booking.
- Utility-based, because when multiple flights satisfy the goal it chooses how to present them - offering the cheapest option first - i.e. it ranks solutions using a utility function (price, preference) rather than presenting an arbitrary goal-satisfying option.

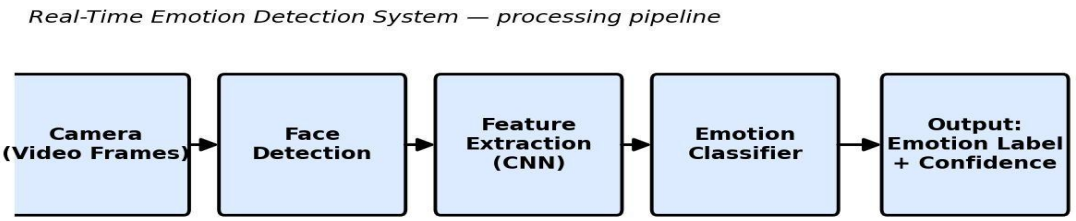
(c) Component Responsible for Each Response

Turn	Message	Likely Component(s)
User 1	"I want to book a flight to Chennai next Friday."	NLU - extracts intent = book_flight and slots {destination: Chennai, date: next Friday}
Chatbot 1	"Sure! What time would you like to depart, and do you have a preferred airline?"	Dialogue Manager - detects missing slots (time, airline) and chooses the next action (ask); NLG - renders the question in natural language
User 2	"Morning flight, no preference."	NLU - extracts slots {time: morning, airline: none}
Chatbot 2	"I found 3 morning flights to Chennai on Friday. Shall I show you the cheapest option first?"	Knowledge Base / backend flight-search service - retrieves matching flights; Dialogue Manager - decides to propose the cheapest-first strategy; NLG - generates the confirming question

(d) Enhancement Using a Learning-Agent Architecture

Add a Learning Element and Critic around the existing performance element: every completed (or abandoned) booking, along with the options the user actually selected, is logged as training data. The critic compares the outcome against a performance standard (e.g. was the first suggestion accepted, how many turns were needed) and feeds this back to the learning element, which updates the utility function used to rank flights - for example, learning that a particular user consistently prefers a specific airline or price band even when they state "no preference". A problem generator can occasionally offer alternative rankings (e.g. fastest instead of cheapest) to keep exploring and avoid settling on a suboptimal ranking policy.

Question 2: Real-Time Emotion Detection System



(a) Function of Each Pipeline Stage

Stage	Function
Camera (Video Frames)	Captures a continuous stream of raw video frames from the environment - the agent's raw sensory input.
Face Detection	Locates the face region within each frame and crops out the background/irrelevant pixels, isolating the region of interest.
Feature Extraction (CNN)	Converts the cropped face image into a compact numerical feature representation (edges, textures, facial-landmark patterns) that captures expression-relevant information.
Emotion Classifier	Maps the extracted feature vector onto one of the predefined emotion classes (e.g. happy, sad, angry, neutral).
Output: Emotion Label + Confidence	Presents the final decision to the user/application as a label together with a confidence score, and can trigger downstream actions.

(b) Likely AI/ML Technique at Each Stage

- Face Detection: a fast detector such as Viola-Jones/Haar cascades, HOG + SVM, or a lightweight deep detector (e.g. MTCNN) suited to real-time use.
- Feature Extraction: a Convolutional Neural Network (CNN) acting as a learned feature extractor / embedding generator.
- Emotion Classifier: a softmax output layer on the CNN (or a separate classical classifier such as SVM/Random Forest trained on the CNN features) performing supervised multi-class classification.
- Output stage: a confidence/probability threshold and label look-up applied to the classifier's output vector.

(c) PEAS Description

- Performance measure: classification accuracy, real-time latency (frames processed per second), robustness to lighting/pose changes, low false-detection rate.
- Environment: live video of a person's face; partially observable (only the visible face, not internal emotional state), continuous, dynamic (lighting/movement changes between frames).
- Actuators: display of the emotion label and confidence score; optional trigger of a downstream alert/response.
- Sensors: the camera feed (video frames).

(d) Agent Type and Justification

This is best classified as a model-based reflex agent. It does not pursue an explicit multi-step goal or plan ahead (ruling out goal-based/utility-based classification); each output is essentially a direct reaction to the current frame. However, a pure simple-reflex agent is not quite sufficient because stages such as face detection/tracking typically rely on an internal model of the face's expected location and appearance built from recent frames (to keep tracking a moving face smoothly and to interpret features in context), so the agent maintains a limited internal state - the hallmark of a model-based reflex agent.

Question 3: Expert System Shell for Medical Diagnosis

(a) Type of Chaining and Justification

Forward chaining is used. Reasoning starts from the given facts (fever = yes, cough = yes, body_ache = yes) and rules are fired whenever their IF-conditions are satisfied, progressively deriving new facts: R1 fires first because fever and cough are already known, deriving suspect = flu; this newly derived fact then satisfies R2 together with body_ache = yes, deriving diagnosis = Influenza. This data-driven, "facts trigger rules" pattern - working forward from what is known towards a conclusion - is the defining behaviour of forward chaining, as opposed to starting from a hypothesis and working backward.

(b) Expert-System-Shell Components Illustrated

Component	Evidence in the Problem
Knowledge Base	The rule base (R1, R2, R3) - the IF/THEN production rules encoding medical knowledge.
Working Memory / Input Facts	fever = yes, cough = yes, body_ache = yes - the current case data supplied to the shell.
Inference Engine	The reasoning trace itself - the mechanism that matches facts to rule conditions, resolves which rule fires (R1 then R2), and updates working memory with derived facts.
Explanation Facility	The generated sentence "Diagnosis = Influenza because fever and cough indicated flu, and flu together with body ache confirmed Influenza" - a plain-language justification of the reasoning chain.

(c) Knowledge Acquisition for a New Dengue Rule

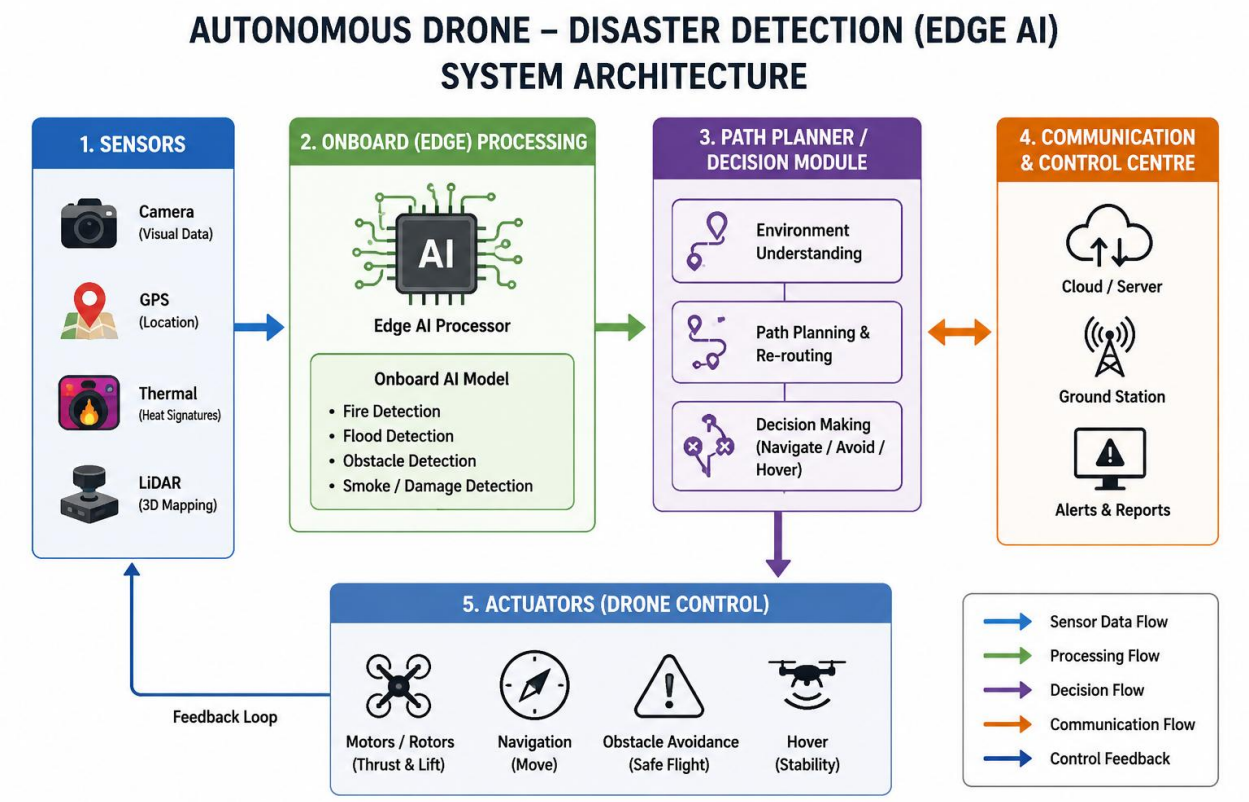
Because the knowledge base is stored separately from the inference engine, new knowledge can be added declaratively without touching the reasoning mechanism. The process would be:

- Interview a medical domain expert to identify the symptom pattern for dengue (e.g. fever, rash, joint pain, low platelet count).
- Express this pattern in the shell's existing IF-THEN syntax, for example: R4: IF fever = yes AND rash = yes AND joint_pain = yes THEN suspect = dengue, and R5: IF suspect = dengue AND platelet_count = low THEN diagnosis = Dengue.
- Add the new rules to the rule base (knowledge base) - no change is needed to the inference engine, since it already knows how to match any IF-condition against working memory.
- Validate the new rules against sample/historical cases (including the existing flu/measles cases) to confirm no unwanted rule conflicts or premature firings are introduced, then activate the rules for live use.

(d) Could Backward Chaining Reach the Same Diagnosis?

Yes. Backward chaining would start from the hypothesis diagnosis = Influenza and search for a rule that concludes it - R2. It checks R2's conditions: body_ache = yes is already a known fact, but suspect = flu is not yet known, so it is set as a sub-goal. The engine then searches for a rule concluding suspect = flu and finds R1, whose conditions (fever = yes, cough = yes) are both known facts, so R1 succeeds and suspect = flu is proven. This satisfies R2, so diagnosis = Influenza is confirmed. Backward chaining therefore reaches the same conclusion, simply by working goal-to-facts instead of facts-to-goal; the two directions coincide here because the rule chain is a single, unambiguous path with no competing rules that would require extra backtracking.

Question 4: Autonomous Drone for Disaster Detection (Edge AI)



(a) PEAS Description

- Performance measure: accuracy/speed of hazard (fire/flood/obstacle) detection, safety (zero collisions), area coverage achieved, timeliness of alerts sent to the control centre, battery/energy efficiency.
- Environment: outdoor disaster-affected terrain; partially observable, continuous, and dynamic (weather, moving hazards, damaged infrastructure); communication with the cloud may be intermittent.
- Actuators: motors/rotors (navigate, avoid obstacles, hover), the alert/data transmission to the control centre.
- Sensors: camera, GPS, thermal sensor, LiDAR.

(b) Agent Type and Justification

The drone is best classified as a utility-based agent (built on top of goal-based planning). It clearly has goals - reach and safely cover the disaster area while detecting hazards - but multiple different flight paths and detection strategies can all satisfy this goal to varying degrees. The agent must therefore weigh competing factors such as coverage thoroughness, obstacle-avoidance safety, detection confidence, and battery consumption, choosing the action that maximises overall utility rather than just any goal-reaching action - for example, deciding whether to divert for a better vantage point at the cost of battery life.

(c) Edge vs Cloud Components and Justification

Runs on the Edge (Onboard)	Runs on the Cloud
Sensors (Camera, GPS, Thermal, LiDAR)	Control Centre (Cloud / Ground Station) - receives alerts
Onboard AI Model (fire/flood/obstacle detection)	Control Centre (Cloud / Ground Station) - receives alerts
Path Planner / Decision Module	Control Centre (Cloud / Ground Station) - receives alerts
Actuators (Motors/Rotors)	Control Centre (Cloud / Ground Station) - receives alerts

On-edge processing is chosen for detection and obstacle avoidance because these functions are latency-critical - avoiding a collision or reacting to a sudden hazard requires millisecond-level responses that a round trip to the cloud cannot reliably guarantee, especially over the unreliable or intermittent network links typical of a disaster zone. Processing on the edge also keeps the drone operational if connectivity is lost entirely, and it drastically reduces the bandwidth needed, since only compact alerts (rather than raw video) are sent to the cloud.

(d) Control Architecture: Reactive, Deliberative, or Hybrid

The drone uses a hybrid control architecture. The Onboard AI Model → Path Planner → Actuators loop for immediate obstacle avoidance behaves reactively, responding directly to current sensor input with minimal deliberation for fast, safe reflexes. At the same time, the Path Planner / Decision Module also performs higher-level, deliberative reasoning - planning coverage routes toward disaster zones and deciding when to alert the control centre - based on an internal model of the mission rather than the immediate percept alone. Combining a fast reactive obstacle-avoidance layer with a slower deliberative mission-planning layer is precisely what defines a hybrid architecture.

Question 5: Deepfake & Face Authenticity Verification System

(a) Likely Input Features

- Blink rate (per 10s): early deepfake generators struggled to reproduce natural, involuntary eye-blinking patterns, so an abnormally low or absent blink rate is a strong authenticity cue - frames 2, 3 and 5 all show very low blink rates (1, 0, 2) alongside high fake confidence.
- Texture/Artifact score: measures blending seams, unnatural skin smoothness, and GAN up-sampling artifacts around the face boundary; higher scores correlate strongly with the fake frames (0.61, 0.83, 0.55) versus the real frames (0.12, 0.09).
- Other features likely feeding a production system, though not shown here, could include optical-flow/temporal consistency between frames, lighting/shadow consistency, and frequency-domain (spectral) artifacts.

(b) Likely Model/Technique Used

A supervised binary classifier trained on these handcrafted and/or CNN-derived features to output a per-frame "fake confidence" percentage - most plausibly a CNN with a sigmoid/softmax output (or a classical classifier such as Random Forest/XGBoost/SVM applied to the extracted blink-rate and texture-artifact features). The output format (a continuous confidence percentage per frame rather than a hard label) indicates a probabilistic classification model.

(c) How the Final Real/Fake Decision Was Derived

Each frame's Fake Confidence is compared against a decision threshold - the Prediction column shows Real for confidences of 8% and 5% (frames 1 and 4) and Fake for confidences of 74%, 91% and 68% (frames 2, 3 and 5), so the threshold sits somewhere in the wide gap between roughly 8% and 55%, most simply 50%. The overall system verdict is then obtained by majority voting across the five per-frame predictions: 3 of the 5 frames were flagged Fake, so the system's aggregate output is Fake, exactly as stated in the log.

(d) Improving Reliability with a Swarm/Ensemble Approach

Instead of relying on a single detector/feature set, deploy an ensemble of independent detectors - for example, a texture-artifact CNN, a separate blink-rate/eye-movement model, an audio-visual lip-sync consistency checker, and a frequency-domain spectral-artifact detector - each acting like one member of a "swarm" independently voting Real/Fake on every frame. Their outputs are then combined through weighted voting or a stacking meta-classifier rather than a single majority-vote count. Because different deepfake-generation techniques tend to leave different tell-tale artifacts, a single model can be fooled while an ensemble is far less likely to be fooled by every member simultaneously, improving overall robustness and reducing false verdicts.